TrendHive E-commerce Platform Documentation

1. Project Overview

TrendHive is a full-featured, multi-vendor e-commerce application designed for scalability and performance. It allows multiple sellers to list products, users to browse and purchase items, and admins to manage the platform.

Key Features

- **Multi-Vendor System**: Sellers can manage their own shops, products, and orders.
- **Advanced Authentication**: Secure Register/Login with **Inline Email OTP Verification**.
- **Real-time Updates**: Socket.io integration for Order Status, Delivery Tracking, and Chat.
- **AI Support**: Gemini-powered customer support chatbot with order context awareness.
- **Performance**: Lazy loading, Redis/Node-cache for API responses, and optimized image delivery via Cloudinary.
- **Logistics**: Delivery Partner OTP verification to ensure secure package handover.

---

2. Technology Stack

Layer	Technology	Usage
Frontend	React (Vite), Tailwind CSS	UI/UX, Responsive Design
Backend	Node.js, Express.js	API Server, Business Logic
Database	MongoDB Atlas (Mongoose)	Flexible Data Storage
Real-time	Socket.io	Live Notifications, Chat, Tracking
Email	Brevo HTTP API	Transactional Emails (OTP, Reset Pwd)
AI	Google Gemini Flash	Customer Support Bot
Storage	Cloudinary	Image Optimization & Hosting

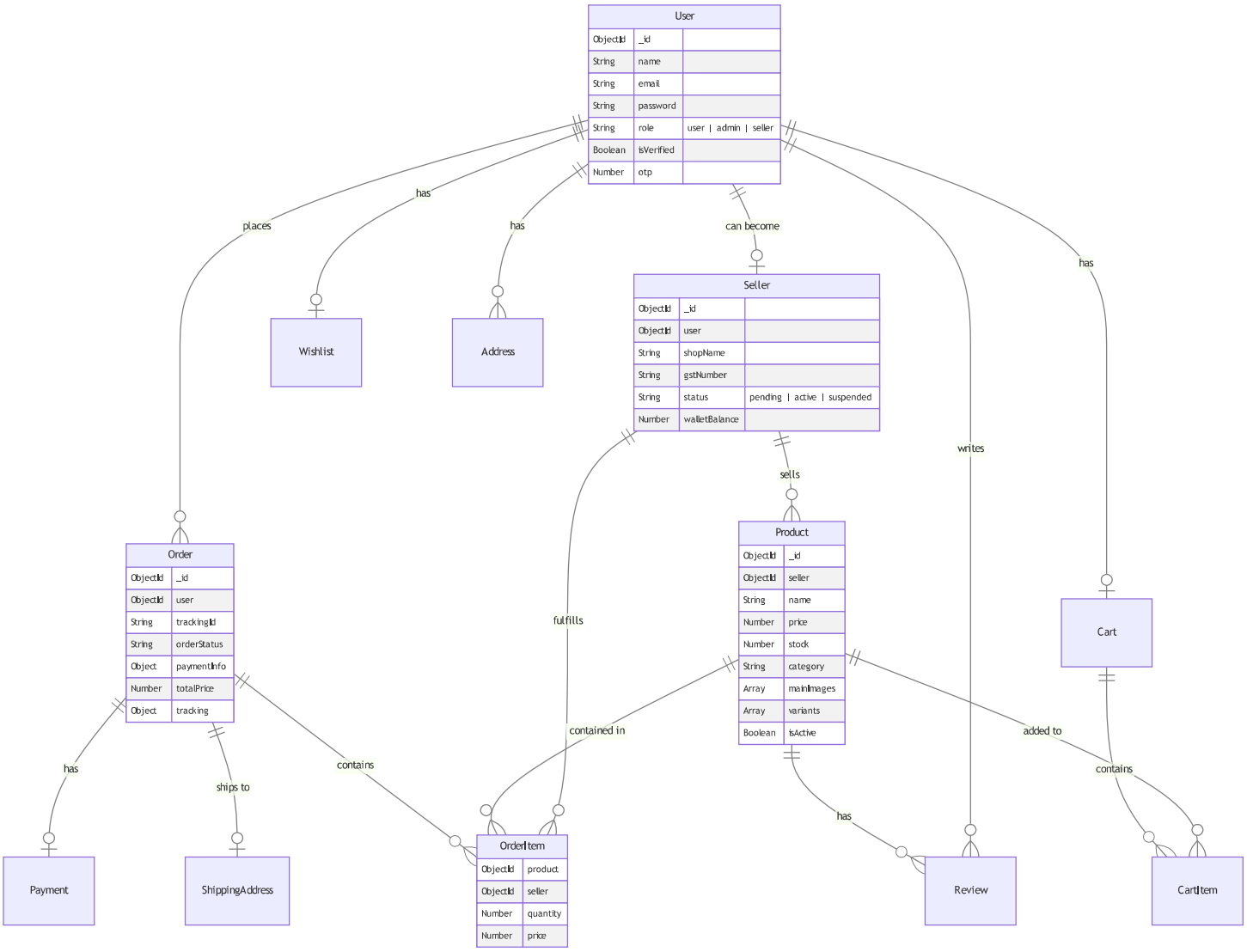
Layer	Technology	Usage
Payments	Stripe / Razorpay / COD	Payment Processing

---

3. System Architecture & Diagrams

3.1 Global ER Diagram

This diagram represents the high-level connection between all major entities in the database, including key field definitions.

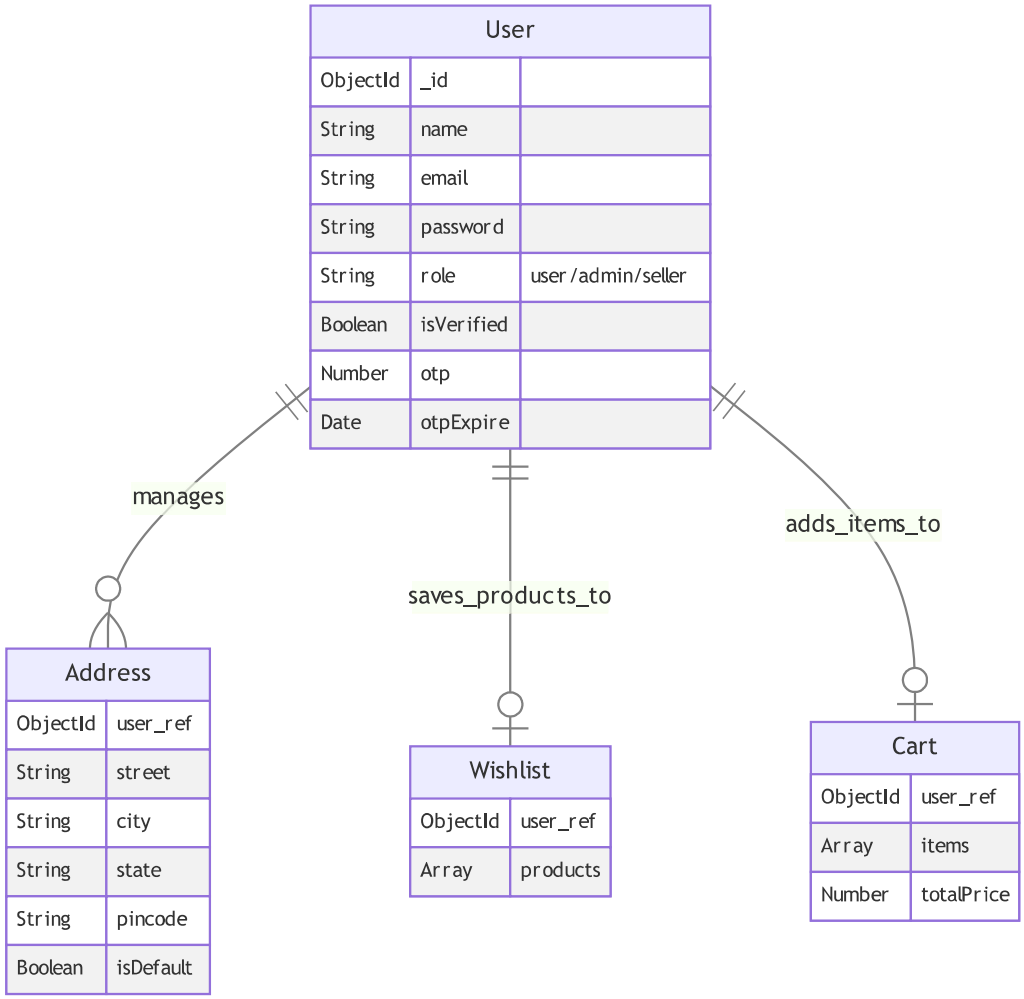


3.2 Authentication & User Profile Sub-System

Focuses on User Identity, Security, and Personalization.

Key Components:

- **User**: Stores auth credentials, role, and verification status.
- **Wishlist**: User's saved items.
- **Address**: Multiple shipping addresses per user.
- **Cart**: Temporary storage for items before purchase.



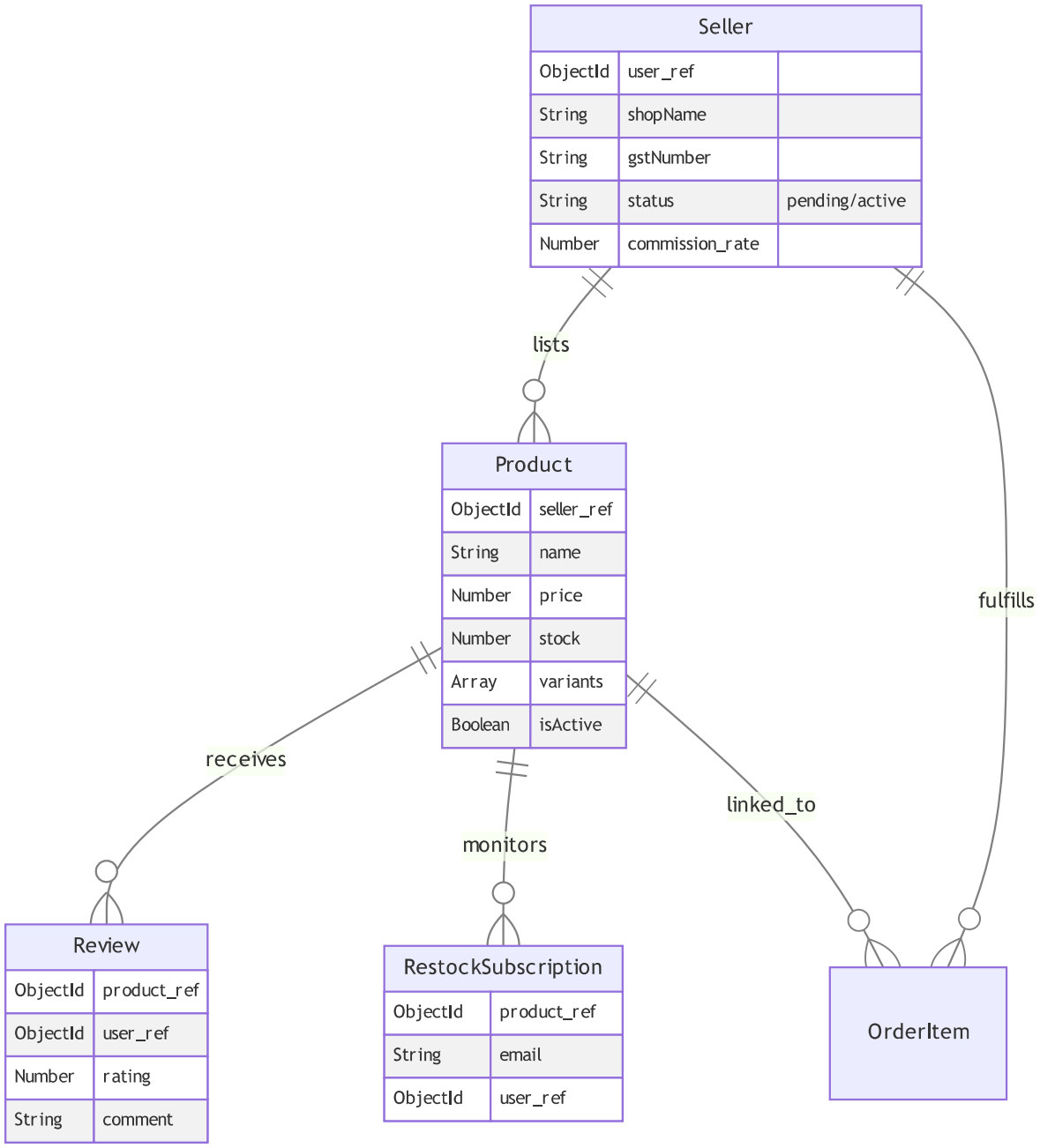
3.3 Product & Inventory Sub-System

Focuses on Sellers, Catalog management, and Stock control.

Key Components:

- **Seller**: Business profile linked to a User.
- **Product**: The core item for sale, including variants (size/color).

- **Review**: User feedback on products.
- **RestockSubscription**: Users waiting for stock updates.



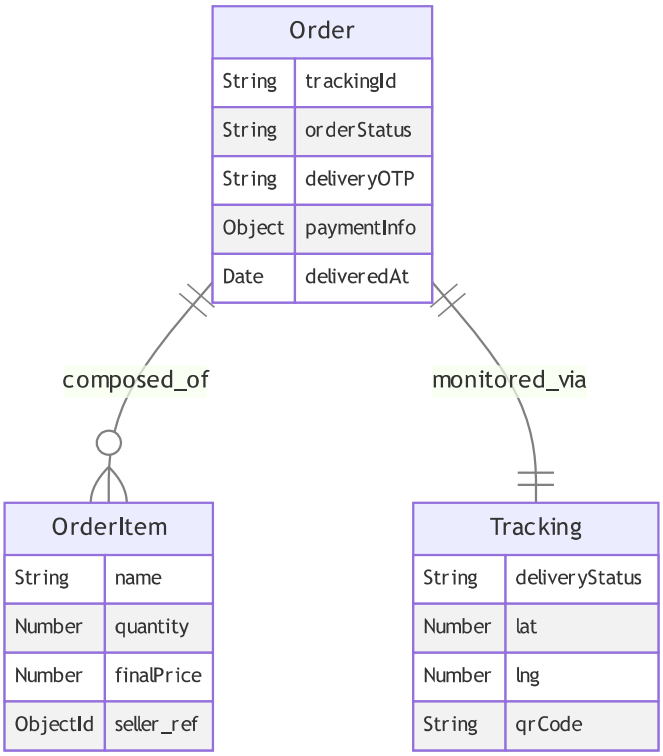
3.4 Order Processing & Logistics Sub-System

The most complex flow involving money and physical goods movement.

Key Components:

- **Order**: The master record of a transaction.

- **OrderItem**: Snapshot of product details at the time of purchase (price/version).
- **Delivery Tracking**: Real-time location and status/OTP.



---

4. Detailed Workflows

A. Secure Registration Flow (Inline OTP)

1. **Input**: User visits `/register`, enters Name, Email, Password.
2. **Request OTP**: Clicks "Send OTP".
   - **Backend**: Checks if email exists. Generates 6-digit `otp`. Hashes password (pre-save). Saves User with `isVerified: false`.
   - **Email**: Sends OTP via Brevo API.
3. **Verify**: Frontend shows OTP input field. User enters code.
   - **Backend**: Matches `req.body.otp` with database `user.otp`. Checks expiry info.
4. **Completion**:
   - Sets `isVerified: true`.
   - Clears OTP fields.
   - Generates JWT Token.

- User is redirected to Profile/Home.

B. Order Fulfillment Cycle

1. **Placement**: User places order. `Order` document created with status `Processing`.
   - Stock is deducted.
   - `Cart` is cleared.
2. **Seller Action**:
   - Seller sees order in "Seller Dashboard".
   - Marks status `Packed -> Shipped`.
3. **Out for Delivery**:
   - Delivery Partner marks status `Out for Delivery`.
   - **System Action**: Automatically generates a 4-digit **Delivery OTP**.
   - **Notification**: Sends OTP to User via Email (Brevo).
4. **Handover**:
   - Partners arrives at doorstep.
   - User shares OTP.
   - Partner inputs OTP in their app.
   - **Verification**: API checks OTP. If match -> Order marked `Delivered`. payment status updated (if COD).

C. Restock Notification Logic

1. **Subscription**:
   - User visits Product Page. Stock = 0.
   - Clicks "Notify Me".
   - Email added to `Product.restockSubscribers` array.
2. **Stock Update**:
   - Seller updates `stock` from 0 to 10.
3. **Trigger**:
   - `ProductController.updateProduct` detects `oldStock === 0 && newStock > 0`.
   - Iterates through subscribers.
   - Sends "Back in Stock" email with product link.
   - Clears subscriber list to avoid spamming on next update.

---

5. API Reference

User Routes (`/api/auth`)

- `POST /send-otp` : Initiate registration.
- `POST /register` : Verify OTP and finalize account.
- `POST /login` : Authenticate and receive cookie.
- `PUT /password/update` : Change password.

Product Routes (`/api/product`)

- `GET /` : List all products (supports filtering/pagination).
- `POST /` : Add new product (Seller only).
- `PUT /:id` : Update product (triggers restock logic).
- `POST /restock/subscribe` : Add user to waitlist.

Order Routes (`/api/order`)

- `POST /new` : Create order.
- `PUT /:id/status` : Update status (Admin/Seller).
- `POST /:id/delivery/otp` : Trigger manual OTP (if needed).
- `POST /:id/delivery/verify` : Verify OTP to complete delivery.

Support Routes (`/api/support`)

- `POST /chat` : Send message to AI Bot.
- `GET /history` : Fetch previous conversation.